

AI WeatherWise – Project Document

weather forecasts and insights

Project Overview

The AI Weather Wise is a robust RESTful backend platform built with **Node.js** and **Express.js**, utilizing **MongoDB** and **Mongoose ODM** for data persistence. It delivers real-time weather metrics integrated with **Google Gemini AI** to provide intelligent summaries and personalized recommendations based on live environmental conditions.

The platform features secure JWT authentication, bcrypt password hashing, and a comprehensive suite of endpoints for managing favorite locations and fetching weather insights. It includes a resilient fallback mode ensuring API functionality even without external API keys, alongside centralized error handling and request sanitization.

Team Members

Team ID:SWTID-2026-4614

Class: III B.Sc Computer Science, Shift – I

TEAM MEMBER	E-MAIL ID	NM-ID
A Deepak	deepakdhanush439@gmail.com	4F972129CD6EE7EA4154CE05AE457D9B
P Deepak Chakaravarthi	deepakdeepak10818@gmail.com	790C2DD82B7CDCEB9E9459AE3B8596C8
Dinesh G	dineshg862007@gmail .com	2CB9C858CB046BE896B421354D8800B6
T K Dinesh	tkdineshtkdinesh3@gmail.com	C094ECDEC80C3A08DD341019D681E3F0

Phase 1

Brainstorming & Ideation Phase

Phase 1: Brainstorming & Ideation Phase

Background

Alex is a travel enthusiast who frequently visits different cities and needs a streamlined way to track local weather conditions and receive AI-driven activity suggestions based on current forecasts.

Problem

- Difficulty tracking and managing multiple favorite weather locations efficiently.
- Lack of personalized recommendations for outdoor activities or clothing.
- Need for natural language weather summaries rather than just raw data metrics.
- Absence of a secure system to store and sync preferences across devices.

Solution

The AI WeatherWise provides a centralized service that handles weather data retrieval and AI orchestrations. It manages a secure User and Location database, allowing for customized insights and automated recommendation workflows through Google Gemini.

Usage

- **User Usage:** Manages favorite cities (CRUD), fetches real-time weather, and requests AI insights for recommendations and summaries.
- **Developer/Admin Usage:** Monitors API health, manages global user registrations, and verifies weather/AI service integrations.
- **Weather Application User:** Interfaces with the public weather endpoints to consume current metrics without requiring an account.

Outcome

By leveraging AI and centralized data management, users spend less time deciphering raw metrics. The system ensures data integrity via Mongoose schemas and provides high-value, context-aware weather advice through Gemini integrations.

Phase 2

Requirement Analysis Phase

Phase 2 : Requirement Analysis Phase

1. Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations).
- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Gemini AI SDK:** Integration using @google/generative-ai for natural language insights.
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

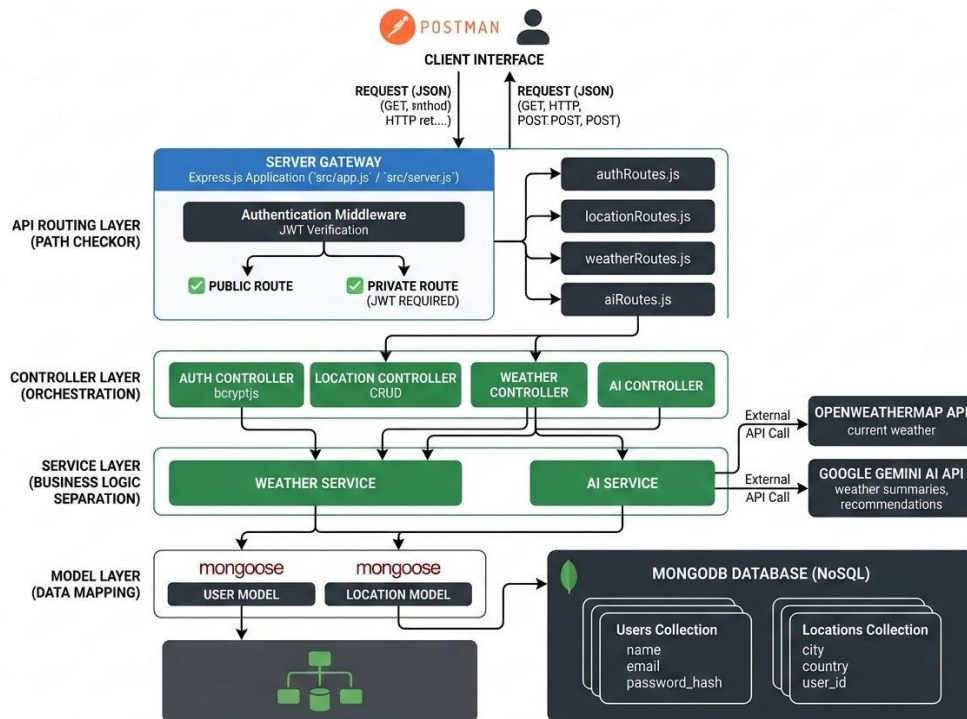
2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

Phase 3

Project Design Phase

Phase 1: Project Design Phase



The AI WeatherWise application follows the MVC pattern to decouple business logic from data management. The Controller layer orchestrates weather fetching from OpenWeatherMap and generation of AI insights via Gemini, while the Model layer manages User and Location entities.

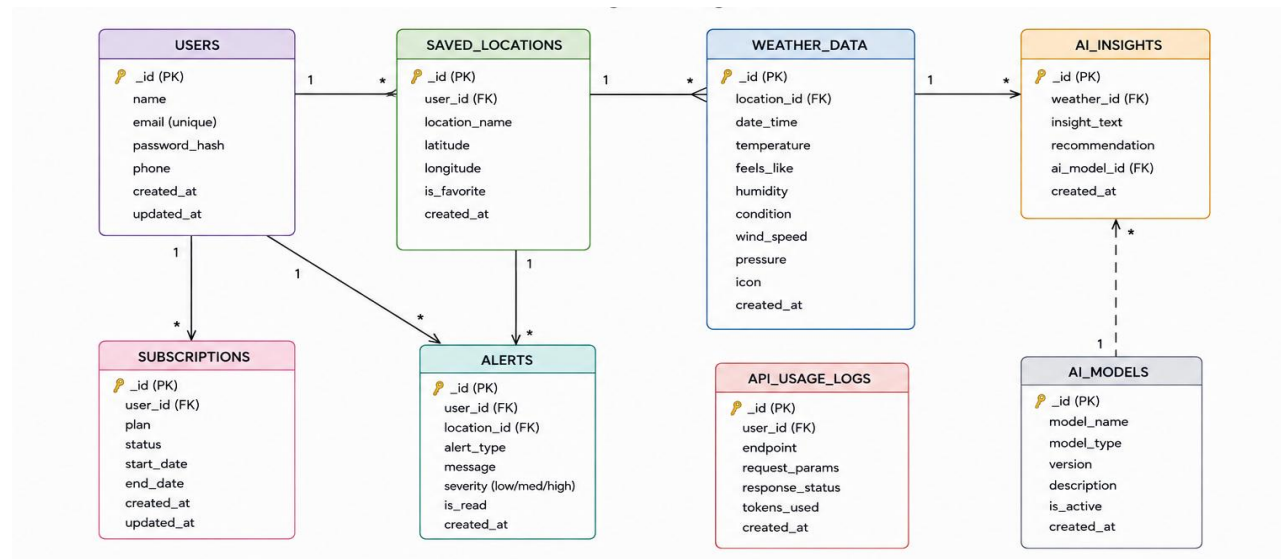
Components Definition

- **Interface:** API tools like Postman query distinct route paths to extract JSON entities.
- **Express Server Gateway:** Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads, and handles CORS headers.
- **Authentication & Role Matrix Middleware:** Extracts tokens from headers, validates user context signatures, and evaluates if the specific role satisfies authorization thresholds for guarded resource paths.
- **Granular Services (Logic Modules):** Enforces isolated domain rules for blog lifecycles (Draft, Pending Approval, Scheduled, Published), comment queues, and system reporting indicators.
- **Database Interface Layer:** Mongoose translates operational application logic into performant native MongoDB queries.
- **AI Service Integration:** Leverages @google/generative-ai to generate weather summaries and personalized recommendations based on live temperature and conditions.

Phase 4

Project Planning Phase

Phase 4: Project Planning Phase



The data architecture manages User profiles and their associated favorite Locations through logical references:

1. User Entity

- _id (Primary Key - ObjectId)
- **name** (String, Required)
- **email** (String, Required, Unique)
- **password** (String, Hashed, Required)
- **role** (String, Default: "reader")

2. Location Entity

- _id (Primary Key - ObjectId)
- **user** (Foreign Key referencing User Collection)
- **city** (String, Required)
- **country** (String, Required)
- **createdAt** (Date, Default: Now)

Key Structural Mappings

- **User to Location (One-to-Many):** A registered user can save multiple favorite locations.

Key Features

- **Role-Based Route Guarding:** Strict token processing blocks unauthorized tiers from modifying core settings.
- **CRUD Blog Execution Engine:** Complete schema management over content definitions, status states, and asset assignment paths.
- **Categorization System:** Native tag indices facilitate filtering and querying for enhanced discoverability.
- **Comment Moderation Logic:** Protected entry points for updating moderation flags (approved, pending, spam).
- **Sanitization Safeguards:** Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections.

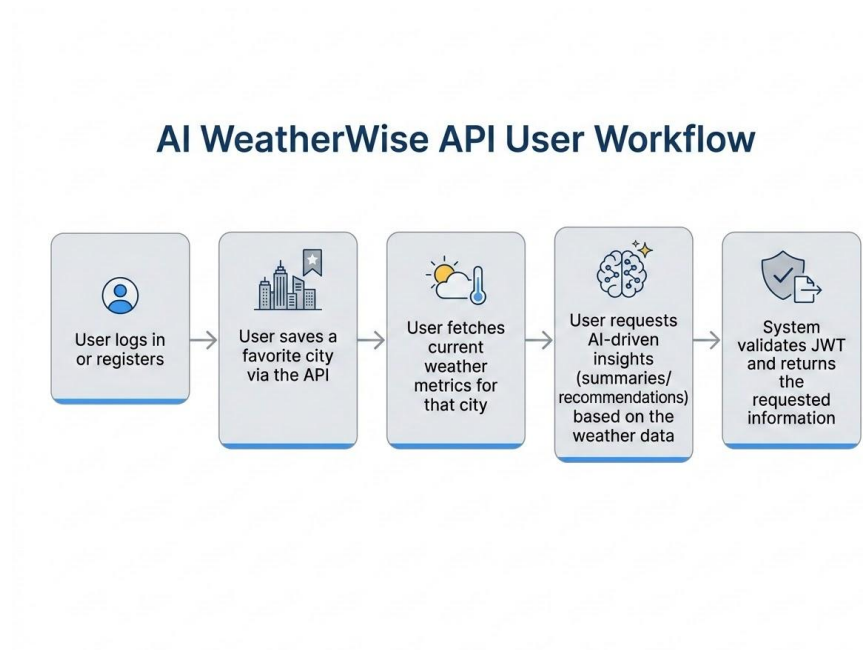
Roles and Responsibilities

- **Admin:**
 - Responsible for user account management, including creating, deleting, or suspending accounts.
 - Oversight of API health and system logs to monitor performance.
 - Monitoring and managing database integrity.
- **Registered User:**
 - Responsible for creating and maintaining a list of favorite locations.
 - Initiating requests for real-time weather data.
 - Interacting with the Gemini AI service for personalized weather summaries and activity recommendations.
 - Managing their own authentication credentials, such as updating profile details.

User Flow:

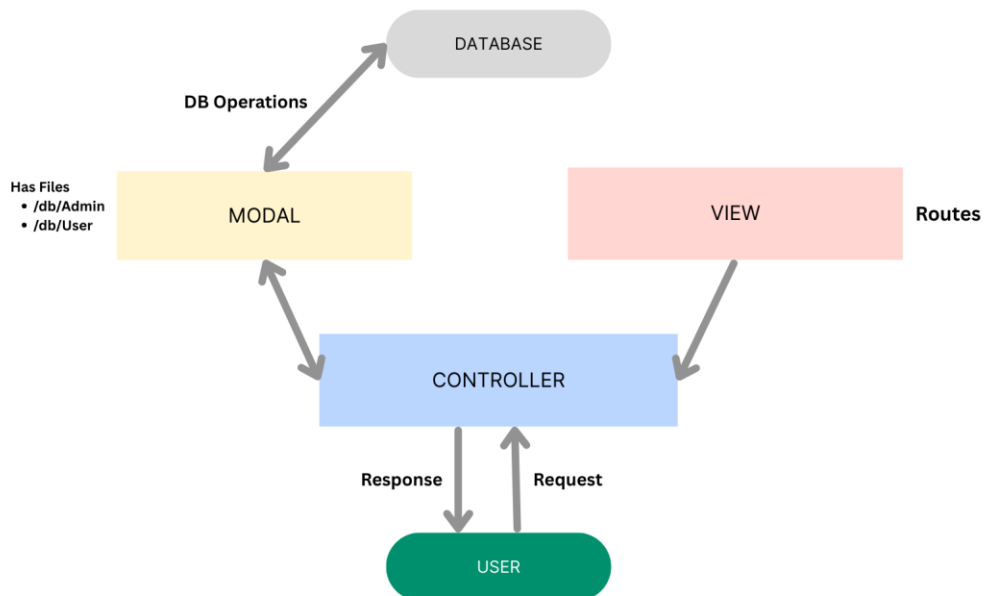
1. User logs in or registers.
2. User saves a favorite city via the API.
3. User fetches current weather metrics for that city.
4. User requests AI-driven insights (summaries/recommendations) based on the weather data.

5. System validates JWT and returns the requested information.



MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



Model Layer (Data Blueprint)

Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets default states, indexing rules, and transformation parameters before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from the execution routes, checks structural request constraints, delegates processing workflows directly to models, and packages the resultant output into standard JSON return arrays.

View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API **Routing Layer**. It links network requests on specified paths directly to their associated controller handler logic maps.

Phase 5

Project Development Phase

Phase 5: Project Development Phase

Creating the Project Folder

- Create a new folder named **AI Weatherwise**
- Open the project folder in **Visual Studio Code**.
- Initialize the project using Node.js and install the required dependencies.

Project Initialization

Open the project folder in the VS Code terminal and execute:

```
npm init -y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer  
@google/genai
```

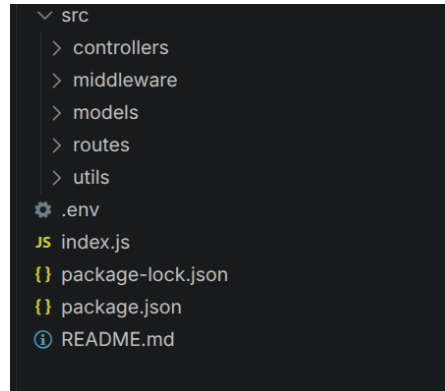
Install Nodemon for development:

```
npm install --save-dev nodemon
```

Create the Following Files

- index.js
- .env
- package.json
- .gitignore

Create the Following Folder Structure



Environment Configuration

Create a **.env** file in the project root and configure the required environment variables.

PORT=5000

MONGO_URI=your_mongodb_connection_string

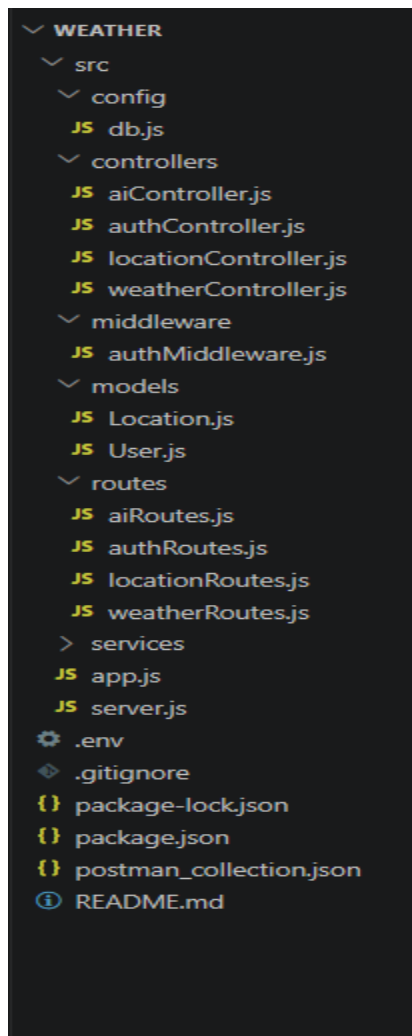
JWT_SECRET=your_jwt_secret

GEMINI_API_KEY=your_gemini_api_key

The AI weatherwise backend is developed using **Node.js**, **Express.js**, and **MongoDB**, following a modular architecture. The application separates routing, controllers, middleware, models, and utility modules to improve code maintainability and scalability. The backend exposes secure REST APIs for authentication, study material management, AI-powered content generation, and administrative operations.

Backend

Structure:



Core System Script Implementations

1. Application Entry Point (**index.js**)

The **index.js** file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.

```

const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
```

API Testing Demo :

1. Configure MongoDB:

- Install Mongoose.
In your Node.js project directory, run:
npm install mongoose
- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions

Location Schema:

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;
```

User schema:

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\\w+([\\.-]?\\w+)*@\\w+([\\.-]?\\w+)*(\\.\\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```

Db connection:

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

GitHub: <https://github.com/deepakdhanush439-oss/AI-WEATHER-WISE.git>

Phase 6

Project Testing Phase

Phase 6 :Project Testing Phase

The AI weatherwise backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables are configured correctly. The backend exposes REST APIs that can be tested using Postman or Thunder Client.

Running the Backend Server

Install all required dependencies:

```
npm install
```

Start the development server:

```
npm start
```

After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

API testing:

Register API:

The screenshot shows a Postman interface for a POST request to `http://localhost:5001/api/auth/register`. The request body is a JSON object with the following fields:

```
{  "name": "John Doe",  "email": "john@mail.com",  "password": "123456"}
```

The response status is 201 (Created). The response body is a JSON object with the following fields:

```
{  "success": true,  "data": {    "_id": "6a4391c99e4426c279d7e4fb",    "name": "John Doe",    "email": "john@mail.com",    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZlNDM5MmM5OWU0NDIyZi13OWQ3ZTRmYiIsIm1hdCI6MTc4MjgxdzEyOSwiZmxwIjoxNzg1NDAlMTI1fQ.7SKxx8zq11JPRb111WN1pR_EyGjdGN-npRBYitCNekk"  }}
```

Login API:

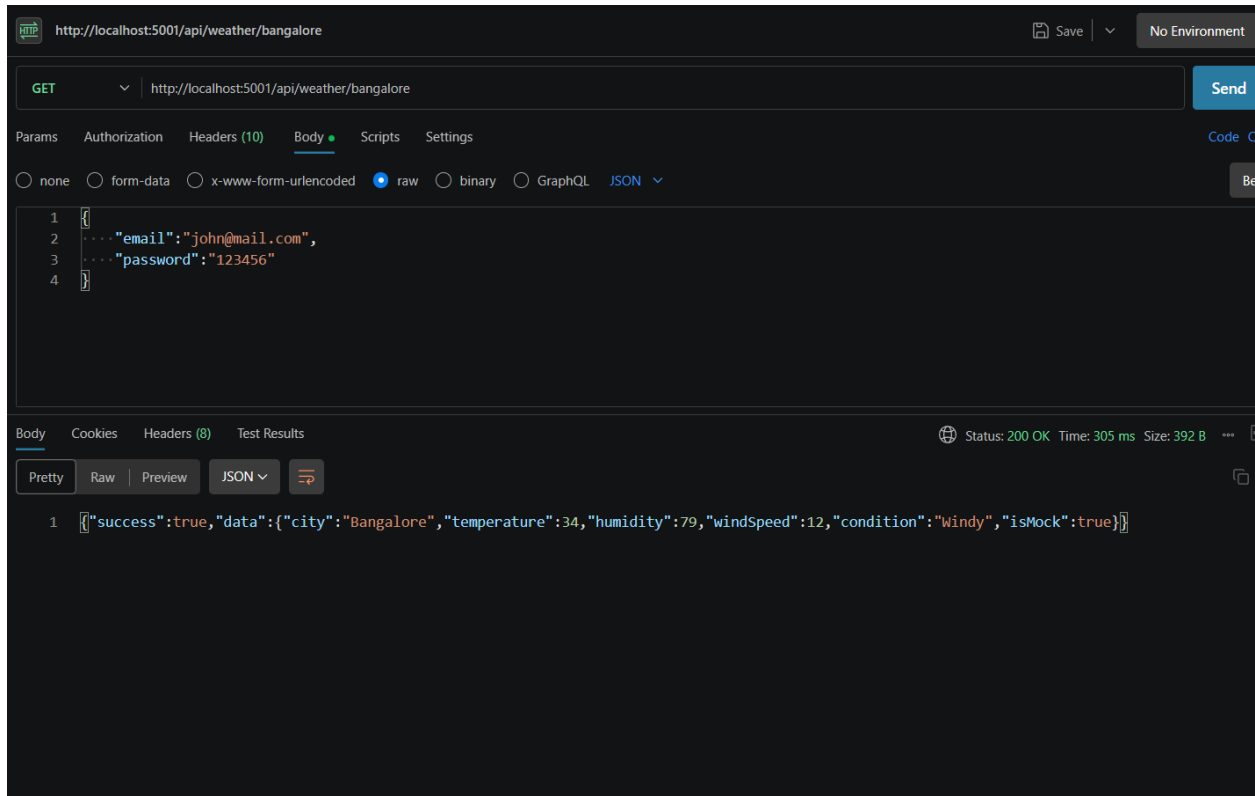
The screenshot shows a Postman interface for a POST request to `http://localhost:5001/api/auth/login`. The request body is a JSON object with the following fields:

```
{  "email": "john@mail.com",  "password": "123456"}
```

The response status is 200 (OK). The response body is a JSON object with the following fields:

```
{  "success": true,  "data": {    "_id": "6a4391c99e4426c279d7e4fb",    "name": "John Doe",    "email": "john@mail.com",    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZlNDM5MmM5OWU0NDIyZi13OWQ3ZTRmYiIsIm1hdCI6MTc4MjgxdzEyOSwiZmxwIjoxNzg1NDAlMTI1fQ.7SKxx8zq11JPRb111WN1pR_EyGjdGN-npRBYitCNekk"  }}
```

Weather API:



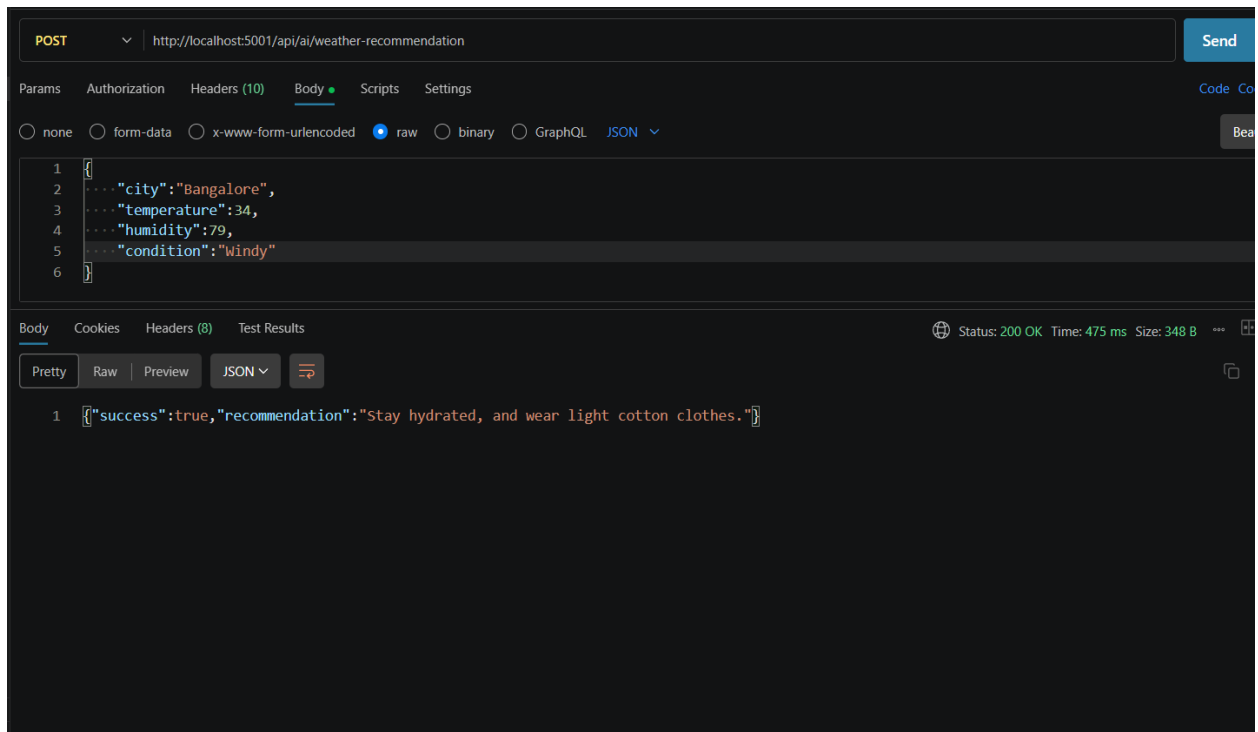
REST client interface showing a GET request to `http://localhost:5001/api/weather/bangalore`. The request body is a JSON object:

```
{  "email": "john@mail.com",  "password": "123456"}
```

The response is a JSON object:

```
{  "success": true,  "data": {    "city": "Bangalore",    "temperature": 34,    "humidity": 79,    "windSpeed": 12,    "condition": "windy",    "isMock": true  }}
```

Recommendation API:



REST client interface showing a POST request to `http://localhost:5001/api/ai/weather-recommendation`. The request body is a JSON object:

```
{  "city": "Bangalore",  "temperature": 34,  "humidity": 79,  "condition": "windy"}
```

The response is a JSON object:

```
{  "success": true,  "recommendation": "Stay hydrated, and wear light cotton clothes."}
```

Phase 7

Project Documentation Phase

Phase 7 : Project Documentation Phase

Phase	Covers
1. Brainstorming & Ideation	Weather forecasting problem & proposed AI solution
2. Requirement Analysis	Software & hardware requirements
3. Project Design	Architecture, ER diagram & MVC
4. Project Planning	Project modules & development structure
5. Project Development	Backend, database, APIs & AI
6. Project Testing	API testing & validation
7. Project Documentation	Code structure, schemas & endpoints
8. Project Demonstration	Working demo & API execution

- **Problem & Requirements** – Explains the weather information problem, proposed AI WeatherWise solution, and required hardware/software.
- **Design & Architecture** – Describes the layered architecture, ER/database structure, MVC pattern, user roles, and system relationships
- **Development & Implementation** – Covers project setup, Node.js/Express backend, MongoDB/Mongoose schemas, authentication middleware, weather APIs, favorite locations, and AI functionality.
- **Testing & Demonstration** – Covers running the API and testing registration, login, weather information, favorite locations, and AI-based weather assistance using API testing tools.& **Requirements** – Explains the customer-support problem, proposed AI FAQ solution, and required hardware/software.

Phase 8

Project Demonstration Phase

Phase 8: Project Demonstration Phase

The project demonstration video presents the complete working process of the AI WeatherWise backend application. The demonstration begins with an explanation of the project structure, major source files, technologies used, and the functionality of each backend component.

Code Explanation

The video explains the main implementation components, including:

- Node.js and Express.js – Used to create and manage the REST API server.**
- MongoDB and Mongoose – Used for database storage and schema management.**
- Authentication Middleware – Demonstrates JWT-based authentication and protected API access.**
- User and Favorite Location Models – Explains the database schemas and relationships used to manage user data and saved locations.**
- API Routes – Covers authentication, weather information, favorite location management, and AI-related routes.**
- Weather API Integration – Demonstrates how external weather data is retrieved and processed for users.**
- AI Integration – Demonstrates how AI is used to provide weather summaries and personalized weather recommendations.**
- Error Handling & Validation – Explains how invalid requests, authentication errors, and database/API errors are handled.**

API Testing

The second part of the demonstration focuses on testing the implemented APIs using Postman/Thunder Client. The following operations are demonstrated:

API Operation	Purpose
User Registration	Creates a new user account
User Login	Authenticates the user and generates a JWT token
User Profile	Retrieves the authenticated user's profile
Add Favorite Location	Saves a location to the user's favorite locations
Get Favorite Locations	Retrieves the user's saved locations
Current Weather	Retrieves current weather information for a selected city
AI Weather Summary	Generates an AI-based summary of the weather
AI Weather Recommendation	Provides AI-based weather recommendations

The API requests, request bodies, authentication tokens, and corresponding responses are shown during the demonstration. This confirms that the backend components are properly connected and the major functionalities of the AI WeatherWise application are working as expected. The project documentation also provides separate links for the API testing video and project demonstration.

Demo Video Link:

<https://drive.google.com/file/d/1b9DnvVokSZSs0F2gUH1QoKr0M1VxZxYd/view?usp=drivesdk>